Character Animation for Operation Code Rescue

Complete, Beautiful, Unique Characters in Unreal Engine 5.7 on Apple Silicon

Operation Code Rescue Prepared with Claude

June 25, 2026

Contents

0.1 1. Introduction and Current State

Operation Code Rescue is a first-person, post-apocalyptic survival-horror game that teaches real programming (Java, C, C++, Python, MATLAB) by having the player solve terminal puzzles while rescuing survivors from zombies across a 465-city campaign. It is built in **Unreal Engine 5.7**, developed solo on macOS (Apple Silicon / Metal), and currently comprises roughly 31,700 lines of C++. This chapter documents the full path from the project's present character state

to commercial-release-quality animated characters, with every recommendation tailored to this codebase, this engine version, and the realities of solo development on Apple Silicon.

The current state is deliberately minimal. The player pawn (`ACodeRescueCharacter`) drives a `UCameraComponent` for the first-person view but has no wired first-person arms mesh or animation. The zombie base class (`ACodeZombieActor`) derives from `ACharacter` and exposes optional, mostly-unassigned slots — a `USkeletalMesh* ProfessionalZombieMesh` and a `TSubclassOf<UAnimInstance> ProfessionalZombieAnimClass` — alongside `UStaticMeshComponent Body` and `Head` primitives that serve as fallback geometry, switched by an `EZombieVariant` enum. The survivor, companion, friendly-NPC, and rescue-squad actors follow the same pattern. Six Fab zombie packs (~5.7 GB across `Content/{DogZombie,Zombie,ZombieFemale,UrbanZombie4,YI_ModularZombies}`) are imported but not animation-wired. There are **no Animation Blueprints, no locomotion Blend Spaces, no retargeting, and no facial animation** anywhere in the project.

The goal is a *refined* survival-horror look — dark, tense, grounded — populated by "complete, aesthetically beautiful, unique individual characters." Achieving that requires a deliberate pipeline rather than ad-hoc asset wiring. The sections below cover character creation, skeletons and rigging, mesh setup and performance, animation authoring and locomotion, retargeting, facial animation, cinematics, a phased roadmap, and validation.

0.2 2. Character Creation Pipeline

Three complementary character sources should feed the project, each chosen for the role it serves best.

0.2.1 2.1 MetaHuman for Human Characters

As of UE 5.6, MetaHuman Creator is fully integrated into Unreal Engine as a plugin rather than a separate cloud app, and **MetaHuman 5.7 brought MetaHuman Creator to macOS and Linux for the first time** [1][2]. This is significant for a solo Mac developer: human roles — survivors, the companion, friendly NPCs, the rescue squad, and even bespoke "hero" zombies — can be authored entirely in-engine on Apple Silicon. To begin, install UE 5.7 with the *MetaHuman Creator Core Data* option enabled, enable the MetaHuman Creator plugin, and create a `MetaHuman Character` asset in the Content Browser; double-clicking opens the MetaHuman Creator asset editor [2].

MetaHuman 5.7 also added major improvements to **body conforming** and more intuitive parametric body controls that make it easier to stay within (or deliberately exceed) human proportions, and it exposed a **Python/Blueprint API covering sculpting, conforming, wardrobe, rigging, and textures**, allowing full character assembly to be batch-processed and scripted [1][3]. For *Operation Code Rescue*, this API is the lever that makes a large, varied cast tractable for one person: a script can stamp out dozens of survivor variants with randomized proportions, wardrobe, and skin tones, then assemble each into a playable Blueprint.

To give each MetaHuman a **unique survival-horror silhouette**, lean on the wardrobe and grooming systems plus sculpting: gaunt or weathered faces, layered scavenged clothing, asymmetric dam-

age, and distinctive head shapes. Silhouette reads first in dark scenes, so prioritize distinct body proportions and headgear over fine texture detail.

0.2.2 2.2 Fab / Marketplace Characters for Zombies

The six imported Fab zombie packs are the right source for the *crowd* of enemies, where MetaHuman fidelity is unnecessary and per-actor uniqueness matters less than variety and silhouette diversity. The `YI_ModularZombies` pack (~4.6 GB) is especially valuable: modular kits allow mixing torsos, limbs, and heads to generate many visually distinct zombies from a shared skeleton, which is ideal for populating a 465-city campaign without authoring each enemy by hand. The standard, dog, female, and urban packs supply additional body archetypes for the `EZombieVariant` system already present in `ACodeZombieActor`.

The work here is not creation but **integration**: confirm each pack's skeleton, build IK Rigs for retargeting (Section 5), author or retarget locomotion and attack animations, and wire the resulting `USkeletalMesh` and `UAnimInstance` into the existing `ProfessionalZombieMesh` / `ProfessionalZombieAnimClass` slots so the primitive fallbacks are replaced.

0.2.3 2.3 Blender → Unreal for Bespoke Characters

For one-off "hero" characters — most importantly the **boss zombie** (`ABossZombieActor`), which deserves a unique, memorable silhouette — a custom Blender authoring path gives the most artistic control. Export discipline is critical because UE uses centimeters (1 unit = 1 cm) while Blender defaults to meters, producing a 100× scale mismatch if left uncorrected [4]. Recommended FBX export practice: select only the Armature and Mesh object types; set Geometry → Smoothing to *Face*; in the Armature tab enable *Only Deform Bones* and disable *Add Leaf Bones*; and rename the armature to `root` so it becomes the root bone on import [4]. Save these as an export preset. Epic's community-maintained **Send to Unreal** addon pushes assets straight into a running editor and removes the manual export/import cycle [4].

The boss should be deliberately *off-silhouette* from the crowd zombies — larger, distorted proportions, distinctive damage or mutation — so the player reads the threat instantly even in low light.

0.3 3. Skeletons and Rigging

0.3.1 3.1 The Shared Skeleton Strategy

The single most important structural decision is to **standardize on the UE5 Mannequin skeleton (Manny/Quinn)** as the project's humanoid hub. MetaHumans already share a compatible skeleton, and the Mannequin is the de-facto target for the vast marketplace and Mixamo animation libraries. Standardizing means animations authored or purchased once can be shared across the player, survivors, companion, and friendly NPCs through retargeting (Section 5), instead of being re-authored per character. Quadrupeds (the dog zombie) and heavily non-human bodies (the boss) need their own skeletons, but every biped should route through the Mannequin.

0.3.2 3.2 Control Rig and Modular Control Rig

Control Rig is UE's in-engine system for rigging and animating characters directly in the editor, bypassing external DCC tools for pose work and Sequencer animation [5]. For *Operation Code Rescue*, Control Rig is the authoring tool for bespoke poses, hand placement on terminals and weapons, and all cinematic animation (Section 7).

Modular Control Rig lets you drag-and-drop pre-made modules — arms, legs, spine, fingers — onto a character; it auto-generates the gizmos and controls needed to start animating immediately, and its standardized components make rigs shareable across characters and projects [6]. This is the right approach for a solo developer rigging multiple distinct bodies (boss, dog, modular zombies): rather than hand-building each rig, assemble modules and reuse them. UE 5.7's refactored **Animation Mode** further streamlines this, with the Pose Library, Tween Tools, Constraints, Selection Sets, and Animation Layers now toggling cleanly and remembering layout state [7].

0.3.3 3.3 IK Rig and the New Skeletal Editor

An **IK Rig** defines solvers and retarget chains on a skeletal mesh and is the prerequisite for retargeting (each source and target needs one) [8]. UE 5.7's IK system added a **Stretch Limb solver** to control squash-and-stretch and improvements to **foot-to-ground contact**, both of which help ground zombie shambles and the boss's weighty movement [7][9]. Separately, UE 5.7's new **Skeletal Editor** brings industry-standard sculpting directly into Unreal — creating and editing morph shapes, bones, and skin weights without round-tripping to Blender [7][9]. For solo work this collapses several tool hops into one editor and is the recommended place to fix skin-weight problems on Fab meshes.

0.4 4. Skeletal Mesh Setup, LODs, Skin Weights, and Apple-GPU Performance

0.4.1 4.1 LODs

A 465-city campaign with crowds of zombies makes Level of Detail mandatory. UE's **Skeletal Mesh Reduction Tool** generates LODs in-editor, each with its own reduction settings [10]. The most relevant knobs for this project are **Max Triangle Count**, **Max Bones Influence** (capping bone influences per vertex — fewer influences are cheaper but stiffer, acceptable at distance), and **Bones to Remove / Bones to Prioritize** so the spine and head keep fidelity while peripheral bones are simplified far from camera [10]. Target an aggressive LOD chain for crowd zombies (e.g., LOD0 hero detail down to a heavily reduced LOD3) and a gentler chain for the player arms and the boss, which are seen up close.

0.4.2 4.2 Skin Weights

Skin Weight Profiles allow a subset of a mesh's skin weights to be replaced and can be assigned per-platform or per-LOD in the Skeletal Mesh Editor's Details panel [11]. This is useful for shipping simplified weights on lower LODs. For Fab meshes with weighting artifacts at joints, the new in-engine Skeletal Editor (Section 3.3) is the fastest fix path.

0.4.3 4.3 Performance on Apple Silicon / Metal

Several macOS-specific constraints shape the budget and **must be designed around**:

- **Nanite and SM6**: Shader Model 6 brings Nanite to M2-and-newer chips, enabled simply by activating the SM6 renderer in project settings — but **SM6 requires macOS 15.x+, and M1 hardware is not supported for Nanite** by Epic [12]. Note that skeletal meshes are traditional (non-Nanite) geometry, so LODs remain the primary skeletal-character optimization regardless of Nanite.
- **Groom / hair strands are not supported on macOS** because Groom requires image-atomic GPU support; **hair cards and hair meshes are supported** [13]. This directly affects the project: the `Content/Grooms` assets and the downloaded MetaHuman hair `.mhpkg` files in `MetaHuman_Downloads/` will **not render as strand-based hair on the Mac target**. Plan to use card/mesh-based hair for all characters, or accept that strand grooms will only work if the game later ships a Windows build.
- **Anti-aliasing**: Temporal Super Resolution (TSR), the default AA, hits hardware limits on Apple Silicon and carries a higher runtime cost there; Epic recommends switching to an alternative AA method in project settings [13]. For a dark, motion-heavy survival-horror game this also reduces TSR ghosting on fast-moving zombies.
- **Ray tracing / Lumen**: hardware ray tracing is not available on macOS, and Lumen uses only the software ray tracer on Apple Silicon [13][12]. Character self-shadowing and contact shadows should be validated under software Lumen, not assumed from Windows behavior.

Set explicit per-frame budgets: a small target for simultaneously-animated crowd zombies at full LOD, with distant enemies forced to low LODs and, where appropriate, animation update-rate optimization (URO) to skip ticks off-screen.

0.5 5. Animation Authoring and Locomotion

0.5.1 5.1 First-Person Arms vs. Full-Body NPCs — Two Different Problems

The player and the NPCs/zombies need fundamentally different solutions, and conflating them is a common mistake.

First-person player arms (`ACodeRescueCharacter`): use a dedicated first-person arms skeletal mesh with a classic, hand-authored **Animation Blueprint** driving a State Machine and Animation Montages. FPS arms are seen extremely close and benefit from tight, deliberately authored idle/walk/sprint sway plus Montages for weapon fire, reload, melee, and — uniquely for this game — **terminal-interaction animations** (reaching to a keyboard, typing). A full Motion Matching rig is overkill for two arms and a weapon; the value of Motion Matching is in full-body locomotion variety, which first-person arms do not show.

Full-body NPCs and zombies: drive these with Animation Blueprints whose AnimGraph combines a locomotion State Machine, **Blend Spaces** for speed/direction, and **Layered Blend Per Bone** for upper-body overrides [14][15].

0.5.2 5.2 The Building Blocks

- **State Machines** switch broad states (idle, locomotion, attack, stagger, death) and manage transitions between them [14].
- **Blend Spaces** mix animations along input axes — e.g., a 1D speed blend (idle → shamble → lunge) or a 2D speed/direction blend for the rescue squad's strafing combat movement [16].
- **Layered Blend Per Bone** blends a separate upper-body animation from a chosen bone (typically `spine_01`), so a zombie can shamble (lower body) while reaching/clawing (upper body), or a squad member can run while aiming [15].
- **Additive animations** layer subtle motion — breathing, weapon sway, flinches — on top of base poses without authoring full clips [15].

0.5.3 5.3 Animation Montages for Discrete Actions

Animation Montages drive non-looping, event-driven actions and are essential for this game's combat and interactions [17]. Montages support **Sections** (named, loopable segments) and **Anim Notifies** (events fired at exact frames) [17][18]. The canonical example maps directly onto *Operation Code Rescue*: a shotgun reload can loop an "insert shell" Section while a Notify increments the ammo count each pass [17]. Use Montages for: player weapon fire and reload (Notify drives the hitscan/projectile and ammo logic), melee swings (Notify opens the damage window), zombie attacks (Notify applies damage at the contact frame), survivor rescue gestures, and the boss's telegraphed special attacks. Notifies are the clean bridge between animation timing and the existing gameplay C++.

0.5.4 5.4 Motion Matching: Status and Recommendation

Motion Matching (the Pose Search plugin) shipped in UE 5.4 and selects poses from a database to produce lifelike movement without large hand-built state machines; it ships with the **Game Animation Sample Project** [19]. In **UE 5.7**, Epic brought a first iteration of **Motion Matching integrated into Choosers** via an experimental Pose Search field, giving per-asset control over which animations are valid for selection rather than only database-level control [9][19]. The 5.7 Game Animation Sample also added the **experimental Mover plugin** (eventual successor to the Character Movement Component) with a new character, ~400 new animations, new walking modes, and a slide mechanic [20].

Recommendation for this project: keep it simple and version-stable. Use a **hand-authored Animation Blueprint with State Machine + Blend Spaces** for zombies, survivors, companion, and squad — this is the proven, low-risk path and matches a solo developer's bandwidth. Treat the **Game Animation Sample as a parts donor**: its hundreds of free, Mannequin-compatible locomotion clips can be retargeted (Section 6) into the project regardless of whether Motion Matching itself is adopted. Defer adopting Motion Matching and the Mover plugin until core gameplay is shipping-stable, since the Mover plugin and the 5.7 Chooser integration are still **Experimental** [20][9] and may shift between engine versions. The rescue squad — allied humans with the richest movement — is the single best candidate for a later Motion Matching upgrade if desired.

0.6 6. Retargeting

Retargeting is the multiplier that lets a solo developer animate a large cast: author or buy a motion once, then share it across many bodies.

0.6.1 6.1 IK Retargeter

The **IK Retargeter** transfers animation between skeletons with different bone counts, names, and orientations by mapping **joint chains** rather than individual bones, optionally preserving hand/foot contact via IK [8]. Because chains are matched (e.g., the whole "arm" chain), a target with more or fewer arm joints than the source still retargets correctly [8]. To create one: in the Content Browser choose Add → Animation → IK Rig → IK Retargeter, pick the source IK Rig, then define matching chains on source and target and run the retarget [8]. UE 5.7 improved retargeting with better foot-to-ground contact, retargeting of squash-and-stretch, and spatially-aware operations that help prevent character self-collision [7][9]. **Auto Retargeting** can auto-generate IK Rig chains for recognized humanoid skeletons as a refinable starting point [21].

0.6.2 6.2 Practical Retargeting Plan

- **Among bipeds** (player full-body proxy, survivors, companion, friendly NPCs, humanoid zombies, rescue squad): build one IK Rig per distinct skeletal mesh, all retargeting to/from the Mannequin hub. A single library of locomotion and action clips then drives every biped.
- **Fab zombie packs**: build an IK Rig for each pack's skeleton and retarget a shared zombie locomotion/attack set onto all of them, so the standard, female, urban, and modular zombies share authored motion despite different meshes.
- **Mixamo and marketplace animations**: Mixamo clips retarget to the Mannequin via IK Rig/Retargeter; the paid **Mixamo Animation Retargeting** marketplace plugin automates the setup (adds a root bone, generates IK Rig and Retargeter assets) and supports root motion and full-body IK if hand-setup proves tedious [22]. Always select the correct **retarget pose** before retargeting, and expect minor manual cleanup afterward [22].
- **Dog zombie and boss**: these non-Mannequin bodies need their own IK Rigs and bespoke or specially-sourced quadruped/creature animation; they will not benefit from the biped library.

0.7 7. Facial Animation and Performance Capture

The game has **230 voiced radio narrations**, which makes facial animation relevant but also raises a hard platform constraint that must be planned around early.

0.7.1 7.1 MetaHuman Animator — and the macOS Limitation

MetaHuman Animator (MHA) generates facial animation from video, depth, or audio, in real time or offline [23]. Its **audio-driven** path is the most relevant for this project: create a `SoundWave` (import the existing narration audio), create a `MetaHuman Performance` asset, set Input Type to *Audio*, pick the SoundWave, and click **Process** to produce Facial Rig animation tracks in Sequencer [23][24]. In principle this could lip-sync on-screen survivor or companion faces to the 230 narrations.

Critical caveat: while **MetaHuman Creator** reached macOS/Linux in 5.7, **MetaHuman Animator support for macOS and Linux is planned for a future release** — it is not yet available on Mac [3]. For a solo Mac-only developer this means audio-driven and capture-based facial animation cannot currently run in the project's primary environment. Practical options, in order of preference: (a) **scope facial animation as a later milestone** pending MHA macOS support; (b) process MHA facial takes on a borrowed/cloud **Windows** machine and import the resulting animation assets back into the Mac project (the generated animation is portable even if the tool is not); or (c) for radio narrations specifically, recognize that the **speaker is off-screen by design** — a radio voice needs no face — so most of the 230 narrations require *no* facial animation at all, and effort should concentrate only on the few on-screen speaking characters.

0.7.2 7.2 Live Link

Live Link streams live facial data into the editor; with the MetaHuman Live Link plugin and the Live Link Face iOS app, a MetaHuman can be animated in real time from a webcam or iPhone [23]. This is a strong, low-cost capture route for a solo developer for the handful of on-screen performances — once MHA's macOS support lands, or via a Windows capture station in the meantime.

0.8 8. Cinematics

Story beats — the campaign intro, survivor-rescue moments, and **boss reveals** — are authored in **Sequencer** with **Control Rig** [5][25]. Drag a Control Rig asset onto a character in a level to open Sequencer with the skeletal mesh and Control Rig tracks attached, then key poses directly [5]. **Layered Control Rig tracks** allow modular, non-destructive edits — a base performance on one track with additive adjustments layered above, ordered to control how each influences the final pose [25]. UE 5.7's Control Rig also gained **world-collision support in physics**, so simulated secondary motion (cloth, hair sway, dangling gear) can interact with level geometry during cinematic playback [7][9] — valuable for grounding a boss reveal in a debris-strewn environment. For this project, a small set of reusable Control Rig setups plus Sequencer is sufficient; outsource motion-capture-heavy cinematic performance only if the budget allows.

0.9 9. Phased Roadmap (Solo-Dev Scoping)

Each phase is scoped for one developer and identifies what to **buy/outsource** versus **author**.

Phase 1 — Prototype Locomotion (replace the fallbacks). Wire one Fab zombie skeletal mesh and a basic Animation Blueprint (idle + walk Blend Space) into `ACodeZombieActor`'s `ProfessionalZombieMesh` / `ProfessionalZombieAnimClass` slots, replacing the primitive `Body`/`Head` geometry. Give the player a first-person arms mesh with an idle and a walk. *Buy*: a small Mannequin-compatible locomotion pack, or harvest the free Game Animation Sample clips. *Author*: the AnimBP wiring.

Phase 2 — The Retargeting Hub. Standardize on the Mannequin skeleton; build IK Rigs for the player proxy, one survivor, and each Fab zombie pack; retarget a shared biped locomotion

set across all of them. Establish the Mixamo/marketplace retargeting workflow. *Buy*: Mixamo Retargeting plugin (optional). *Author*: IK Rigs, retarget poses.

Phase 3 — Combat and Interaction Montages. Author Montages with Notifies for player fire/reload/melee, terminal-typing interaction, zombie attacks, and survivor rescue gestures; bind Notifies to existing gameplay C++. *Author*: all (this is gameplay-critical and should not be outsourced).

Phase 4 — Character Identity (MetaHuman + Boss). Use MetaHuman Creator (now on macOS) and its Python API to generate distinct survivors, the companion, friendly NPCs, and the rescue squad; author the boss in Blender with a Modular Control Rig and bespoke animation. *Outsource* (optional): boss mesh sculpt and signature attack animations. *Author*: MetaHuman assembly scripting, integration.

Phase 5 — Locomotion Polish. Add 2D directional Blend Spaces, Layered Blend Per Bone upper-body overrides, additive idles/flinches, and IK foot placement; evaluate Motion Matching for the rescue squad only. *Author*: all.

Phase 6 — Facial and Cinematics. Add on-screen-character facial animation (via MHA on Windows/cloud, or once macOS support ships, or Live Link), and build Sequencer + Control Rig cinematics for the intro, rescues, and boss reveal. *Outsource* (optional): mocap for key cinematic performances.

Phase 7 — Optimization and Validation. Author LOD chains, skin-weight profiles, and animation update-rate optimization; run the full QA pass in Section 10.

0.10 10. Pitfalls, Performance Budgets, and QA / Validation

Common pitfalls to avoid: - **Blender 100× scale** errors on import — always use the export preset and `root` armature [4]. - Assuming Windows behavior on Mac: **Groom strands, hardware ray tracing, and M1 Nanite are unavailable**; TSR is costly [12][13]. Use hair cards, software Lumen, and an alternative AA. - Treating FPS arms and full-body NPCs as one problem — they need different rigs and different animation strategies (Section 5.1). - Adopting **Experimental** systems (Mover, 5.7 Motion Matching Choosers) on the critical path before they stabilize [20][9]. - Forgetting to set the **retarget pose** before retargeting, which produces broken results [22].

Performance budgets (set and profile against these): cap simultaneously full-LOD-animated zombies; force distant enemies to reduced LODs; apply animation URO off-screen; bound bone-influence counts on lower LODs via the Reduction Tool [10]; and validate frame time on the actual Apple Silicon target rather than a Windows reference.

QA / Validation. Extend the project's automated-check suite with Unreal's **Data Validation** framework. `UObject::IsDataValid` overrides and registered `UEditorValidatorBase` validators are run by the `UEditorValidatorSubsystem`, which can be invoked in-editor and in commandlets/CI [26]. Author validators that assert, for every character actor: a skeletal mesh is assigned (no shipping fallback primitives), an Animation Blueprint class is set, the mesh uses the expected skeleton, LODs exist, and no macOS-incompatible Groom strand asset is referenced on the Mac target. Wiring these into the project's existing validation pass turns "did I forget to assign the

AnimBP on this zombie variant?" from a runtime surprise into a build-time failure — exactly the kind of guard a solo developer relies on across a 465-city, multi-variant cast.

0.11 References

[1] MetaHuman 5.7 is now available — https://www.metahuman.com/releases/metahuman-5-7-is-now-available [2] Creating your MetaHuman in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-your-metahuman-in-unreal-engine [3] MetaHuman 5.7 Release Notes | MetaHuman Documentation — https://dev.epicgames.com/documentation/en-us/metahuman/metahuman-5-7-release-notes [4] FBX Skeletal Mesh Pipeline in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/fbx-skeletal-mesh-pipeline-in-unreal-engine [5] How to Animate with Sequencer | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/how-to-animate-with-sequencer [6] Modular Control Rigs in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/modular-control-rigs-in-unreal-engine [7] Unreal Engine 5.7 Release Notes | Epic Developer Community — https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-5-7-release-notes [8] IK Rig Animation Retargeting in Unreal Engine | Epic Developer Community — https://dev.epicgames.com/documentation/unreal-engine/ik-rig-animation-retargeting-in-unreal-engine [9] Unreal Engine 5.7 is here! Find out what's new — https://www.unrealengine.com/news/unreal-engine-5-7-is-now-available [10] Skeletal Mesh LODs in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/skeletal-mesh-lods-in-unreal-engine [11] Skin Weight Profiles in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/skin-weight-profiles-in-unreal-engine [12] Unreal Engine 5.2 brings native support for Apple Silicon (macOS) — https://www.unrealengine.com/en-US/tech-blog/unreal-engine-5-2-brings-native-support-for-apple-silicon-and-other-developments-for-macos [13] Bringing Unreal Engine on macOS up to feature parity with Windows — progress report — https://www.unrealengine.com/tech-blog/bringing-unreal-engine-on-macos-up-to-feature-parity-with-windowsprogress-report [14] Animation Blueprint Blend Nodes in Unreal Engine | Epic Developer Community — https://dev.epicgames.com/documentation/unreal-engine/animation-blueprint-blend-nodes-in-unreal-engine [15] Using Layered Animations in Unreal Engine | Epic Developer Community — https://dev.epicgames.com/documentation/unreal-engine/using-layered-animations-in-unreal-engine [16] Blend Spaces in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/blend-spaces-in-unreal-engine [17] Animation Montage Overview | Epic Developer Community — https://dev.epicgames.com/documentation/en-us/unreal-engine/animation-montage-overview [18] Animation Notifies in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/animation-notifies-in-unreal-engine [19] Motion Matching in Unreal Engine | UE 5.7 Documentation — https://dev.epicgames.com/documentation/en-us/unreal-engine/motion-matching-in-unreal-engine [20] Explore the updates to the Game Animation Sample Project in UE 5.7 — https://www.unrealengine.com/tech-blog/explore-the-updates-to-the-game-animation-sample-project-in-ue-5-7 [21] Auto Retargeting in Unreal Engine | Epic Developer Community — https://dev.epicgames.com/documentation/unreal-engine/auto-retargeting-in-unreal-engine [22] Mixamo Animation Retargeting 2 — Retarget a Mixamo animation to a UE5 character |

UNAmedia — https://www.unamedia.com/ue5-mixamo/docs/retarget-mixamo-to-ue5/ [23] MetaHuman Animator in Unreal Engine | MetaHuman Documentation — https://dev.epicgames.com/documentation/en-us/metahuman/metahuman-animator [24] Audio Driven Animation | MetaHuman Documentation — https://dev.epicgames.com/documentation/metahuman/audio-driven-animation [25] Animating with Control Rig in Unreal Engine | Epic Developer Community — https://dev.epicgames.com/documentation/en-us/unreal-engine/animating-with-control-rig-in-unreal-engine [26] UEditorValidatorBase | UE 5.7 Documentation (Data Validation plugin) — https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Plugins/DataValidation/UEditorValidatorBase